



UNIVERSITÀ DI PISA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Laurea Triennale in Ingegneria Informatica

**Automazione della configurazione OSPFv3 in reti
IPv6 emulate con Containerlab**

Relatori:

Prof: Enzo Mingozzi

Prof: Antonio Viridis

Candidato:

Daniele Congiusti

ANNO ACCADEMICO 2024/2025

Indice

1	Introduzione	5
1.1	Network management	5
1.2	Applicazione	5
2	IPv6 e OSPFv3	7
2.1	Perché IPv6 ?	7
2.2	Indirizzamento IPv6	7
2.2.1	Tipologie di indirizzi IPv6	7
2.2.2	Notazione degli indirizzi IPv6	8
2.3	OSPFv3	9
2.3.1	differenze con OSPFv2	9
3	ContainerLab	11
3.1	Vantaggi e feature	11
3.2	Creazione topologia di rete	11
3.3	Deploy della topologia	13
4	Back-end in python	15
4.1	Pyeapi client	15
4.1.1	Connect	15
4.1.2	Enable	16
4.2	Classi	17
4.2.1	Path_To_ASBR	17
4.2.2	Route	17
4.2.3	Link	18
4.2.4	Node	19
4.2.5	Area	20
4.2.6	Network	21
4.3	Funzioni getter lsa	22
4.4	File main	23
4.4.1	Dettagli sul nodo scelto	23
4.4.2	Ricostruzione della topologia	24
5	Front-end web-based	35
5.1	Hosting Flask	35
5.2	Vis.js	37
5.3	Interfaccia interattiva	38

5.4	differenze per reti IPv6	38
6	Utilizzo dell'applicazione	43
6.1	Script bash	43
6.2	Immagini dei container	44
6.3	Terminale dell'applicazione	44
7	Conclusioni	47
7.1	Considerazioni sul lavoro svolto	47
7.2	Possibilità future	47

Capitolo 1

Introduzione

1.1 Network management

Il network management è un elemento essenziale per poter gestire le reti informatiche, serve per gestire l'infrastruttura già presente e aiuta ad estenderla per poter offrire connettività e servizi. Altrettanto importante è la gestione delle reti all'interno degli *Autonomous System* (AS), la rete interna di un'organizzazione, nella quale si possono ritrovare una o più sottoreti da dover gestire. Il compito principale del network Management rientra quindi nel garantire sia connettività all'interno dell'AS, sia permettere ai nodi della rete di comunicare con l'esterno. Per supportare questo processo vengono sviluppate applicazioni in grado di recuperare informazioni, sulla rete, e rappresentarle in maniera sistematica e precisa, permettendo agli amministratori di rete di effettuare una migliore gestione ed eventualmente una più rapida risoluzione di problemi di rete.

1.2 Applicazione

Questa applicazione, estensione della versione per OSPFv2, ha lo scopo di ricostruire la topologia di una rete configurata con OSPFv3, facendo ausilio delle informazioni nel database di un router della rete (scelto dall'utente). Per realizzare tutto ciò, il programma recupera gli *lsa* ricevuti dal router scelto, dai suoi vicini¹ e i restanti router della rete, necessari per completare la ricostruzione della topologia. Il programma inizia elaborando gli lsa di tipo 1, per poi procedere con quelli di tipo 2, fino agli lsa di tipo 9². Il vantaggio offerto consiste nel riuscire a recuperare le informazioni intra-area, quelle inter-area (ottenute da altre aree all'interno del AS) e informazioni esterne alla rete dell'organizzazione. Il programma è stato progettato utilizzando il software di emulazione ContainerLab, il quale ha permesso di emulare topologie di rete sfruttando i container Docker, un'alternativa più leggera delle macchine virtuali. Tramite l'utilizzo di un terminale, a cui si è fatto accesso tramite SSH o Docker, è stato possibile effettuare le configurazioni sui nodi (per i router) oppure eseguire l'applicativo (nel caso di endpoint). Oltre alla cli (Command Line Interface) fornita, il software sviluppato dispone anche di un'interfaccia grafica web-based, supportata in back-end da Flask, il quale fornisce un servizio di hosting e di renderizzazione di template, e in front-end dalla libreria Vis.js, in grado di riprodurre lo schema della topologia di rete grazie all'ausilio dei grafi. L'applica-

¹Vicinanza intesa nell'ambito del protocollo OSPF

²Gli lsa di tipo 6 e di tipo 7 non sono stati presi in considerazione per la ricostruzione della topologia

zione che fa uso di IPv4 e OSPFv2, è stata naturalmente adattata per fare uso dei protocolli IPv6 e OSPFv3, tenendo conto delle differenze riscontrate tra le versioni dei protocolli sopra citati. A seguire viene riportata una figura, che illustra un esempio della rappresentazione della topologia di rete realizzata tramite l'interfaccia grafica dell'applicazione, in modo tale da fornire un'idea del risultato finale ottenuto.

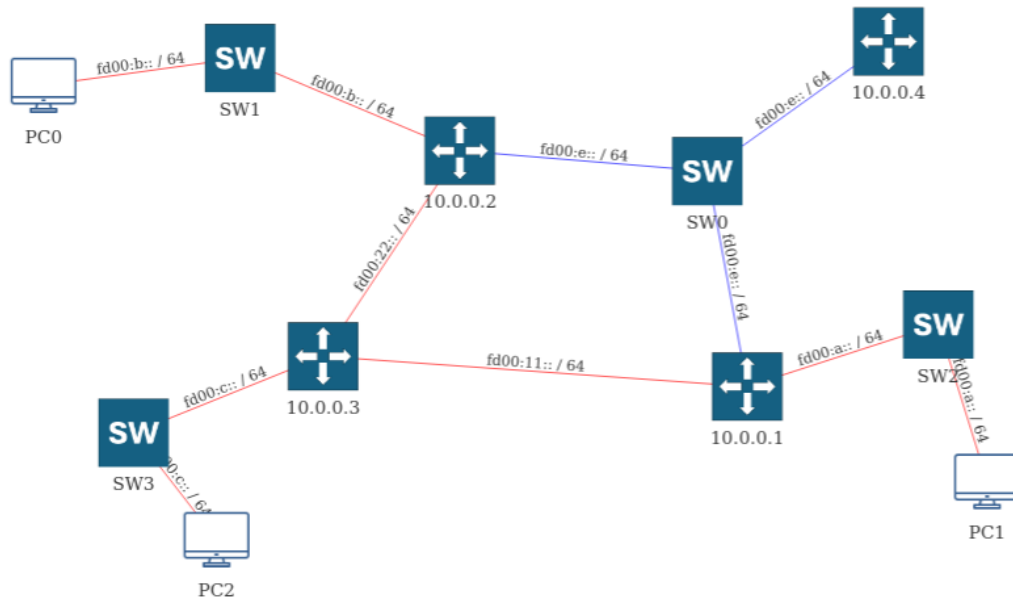


Figura 1.1: Esempio rappresentazione topologia di rete

Capitolo 2

IPv6 e OSPFv3

Il capitolo seguente ha lo scopo di fornire le nozioni minime necessarie per comprendere il funzionamento dell'applicazione e come interpretarne correttamente i dati.

2.1 Perché IPv6 ?

Il protocollo IPv6 risulta il vero successore¹ del protocollo IPv4 , che verrà sostituito con l'evoluzione delle reti informatiche. La necessità di sviluppare un nuovo protocollo è sorta dal momento in cui il quantitativo di dispositivi, che sfruttavano e richiedevano il servizio di connettività , e la presenza di nuove reti stavano portando all'esaurimento degli indirizzi assegnabili. Inizialmente si provò a porre rimedio a questo problema sfruttando ulteriori protocolli, come NAT (Network Address Translation), oppure definendo dei range di indirizzi privati e pubblici, o ancora di progettare le reti con una maschera di lunghezza variabile (VLSM, Variable Length Subnet Mask). Queste soluzioni si sono rivelate essere solo un rimedio temporaneo, che ha rimandato il problema dell'esaurimento degli indirizzi assegnabili. Questi fattori hanno portato alla nascita del protocollo IPv6, che fa fronte alla necessità di indirizzi assegnabili e prevede anche ulteriori funzionalità, precedentemente supportate da altri protocolli (con la versione IPv4).

2.2 Indirizzamento IPv6

Prima di effettuare un'analisi del piano di indirizzamento del protocollo IPv6, si deve esplicitare il formato dell'indirizzo adottato. L'indirizzo IPv6 è composto da 128 bit, espressi in otto gruppi da quattro cifre esadecimali, rendendo così disponibili 2^{128} indirizzi utilizzabili. L'intervallo di indirizzi effettivamente utilizzabili è in realtà ridotto, perché vi sono range di indirizzi riservati e, quindi, non utilizzabili in fase di progettazione. Nonostante questa riduzione, gli indirizzi rimanenti permettono di affrontare il problema sopra discusso.

2.2.1 Tipologie di indirizzi IPv6

Un primo aspetto da analizzare, per comprendere il piano di indirizzamento IPv6, è indicare le 3 tipologie di indirizzi assegnabili:

1. **Unicast:** indirizzi adoperati per identificare univocamente un'interfaccia di un nodo IPv6

¹IPv5 non venne poi adoperato per somiglianza con IPv4

2. **Multicast**: indirizzi che identificano un gruppo di interfacce, che diverranno destinatarie dei pacchetti inviati all'indirizzo Multicast
3. **Anycast**: indirizzi assegnabili a più di un'interfaccia ²

IPv6 permette di configurare più di un indirizzo su un'interfaccia, questo ha portato a stabilire che, per ogni interfaccia IPv6, dovrà essere assegnato almeno un indirizzo Unicast, perché risulta essenziale identificare univocamente l'interfaccia. Un ulteriore aspetto degli indirizzi IPv6 è lo scope, ovvero la visibilità di cui esso dispone. Lo scope di un indirizzo può essere globale o non globale e, come per gli indirizzi privati e pubblici in IPv4, vengono definiti degli intervalli che permettono di associare uno scope all'indirizzo in questione. Per fornire ulteriori dettagli sul funzionamento dell'applicazione si riportano gli scope utilizzati:

- **Link-local address**: indirizzi usati per un singolo link e non routable
- **Unique Local Address**: indirizzi utilizzati all'interno della rete dell'organizzazione, i quali sono univoci a livello globale e non routable
- **Global Unicast Address**: costituisce la categoria di indirizzi globali unicast ³

I valori, quindi, da assegnare agli indirizzi, per avere un determinato scope, sono indicati nella tabella 2.1.

2.2.2 Notazione degli indirizzi IPv6

Come detto precedentemente, gli indirizzi in IPv6 vengono rappresentati da 8 blocchi, ognuno composto di 4 cifre esadecimali (per un totale 128 bit). Si riscontra una differenza importante rispetto alla rappresentazione in IPv4: i blocchi dell'indirizzo sono separati dal carattere ':' e non più dal carattere '.'. Per fare ulteriore chiarezza viene riportato un esempio di indirizzo IPv6: 2001:0db8:0000:0000:0000:ff00:0042:8329

Per evitare errori e migliorare la leggibilità, sono state definite delle regole per la scrittura degli indirizzi IPv6 ⁴:

- qualora nel blocco si presentino degli zeri che precedono altre cifre, diverse da zero, questi non vengono rappresentati
- nel caso in cui un blocco sia composto da 4 zeri viene sostituito con un singolo zero
- Se nell'indirizzo vi sono più blocchi consecutivi composti unicamente da zeri, questi possono essere sostituiti dal doppio semicolon (::)
- le lettere vengono scritte in minuscolo (a,b,c,d,e,f)

Per proporre un esempio di applicazione di queste regole, si riporta l'indirizzo dell'esempio precedente in forma "compressa":

2001:0db8:0000:0000:0000:ff00:0042:8329 ⇔ 2001:db8::ff00:42:8329

Prima di indicare i prefissi associati agli scope, è necessario dire che i 64 bit meno significativi,

²Solamente un'interfaccia sarà scelta, tramite un criterio, come destinataria dei pacchetti

³Nell'applicazione verranno utilizzati per i link punto punto tra ASBR e ISP

⁴Regole dallo standard RFC 5952

dell'indirizzo IPv6, sono in funzione del MAC address dell'interfaccia, per questa motivazione non si vedranno, generalmente, prefissi con valore maggiore di 64. Fatta questa premessa, viene illustrata una tabella con indicati i prefissi e gli scope associati⁵:

Prefisso	Scope
<i>fe80 :: /10</i>	<i>link – local</i>
<i>fd00 :: /7</i>	<i>unique – local</i>
<i>2000 :: /3</i>	<i>global – unicast</i>

Tabella 2.1: Prefissi indirizzi scope

2.3 OSPFv3

Prima di parlare del protocollo OSPFv3, si deve tenere in considerazione che la maggior parte degli aspetti di OSPFv2 sono mantenuti in questa nuova versione, però alcuni subiscono dei cambiamenti, che verranno esposti in seguito.

2.3.1 differenze con OSPFv2

Di seguito verranno indicate le differenze che il protocollo OSPFv3 presenta rispetto alla versione precedente.

Protocollo per-link

Il protocollo OSPFv3 si può definire come protocollo **per-link**. Questo è dovuto al fatto che il protocollo IPv6 associa le interfacce ai link (non alle subnet) e inoltre supporta la presenza di più indirizzi in una singola interfaccia. Di conseguenza, dato il ruolo centrale dei link e anche per rendere più veloce la configurazione, il protocollo opera direttamente sui link, ovvero sulle interfacce, pubblicizzando così tutti i prefissi configurati in esse. Questo costituisce un grosso vantaggio, in quanto si dovranno solamente selezionare le interfacce coinvolte per la scoperta della topologia, senza elencare tutte le subnet configurate nella rete dell'organizzazione.

Rimozione indirizzi

Il protocollo OSPFv3 non prevede indirizzi negli header OSPF, bensì permette di riportarli solo nel payload, richiedendo quindi delle operazioni per il recupero dei prefissi. Nonostante l'utilizzo di IPv6, **Router ID**, **Area ID** e **Link state ID** vengono comunque espressi su 32 bit⁶ utilizzando la dot notation (4 interi separati da '.') e questa scelta implementativa comporta due conseguenze:

1. Quando si trattano i link di tipo *Transit Network* verranno indicati DR (Designated Router) e BDR (Backup Designated Router) tramite i rispettivi Router ID
2. i campi indicati sopra non possono assumere indirizzi IPv6

⁵Prefissi usati anche nei casi di test dell'applicazione

⁶Utilizzato per indicare gli indirizzi delle interfacce di loopback dei nodi

Multiple istanze OSPF

Particolarità del protocollo OSPFv3 è data dalla possibilità di avere più processi OSPF (con diverso ID) sullo stesso link, comportando quindi l'avere due database differenti e due gestioni diverse della topologia⁷. La possibilità di avere più istanze OSPF in un'interfaccia può essere sfruttata per associare il link a più aree dell'AS.

Autenticazione

OSPFv3 offre integrità e autenticazione, sfruttando IP authentication header oppure IP Encapsulating Security Payload

Cambiamenti LSA

Ci sono dei cambiamenti per quanto riguarda i tipi già presistenti degli LSA (Link State Advertisement): il tipo Summary link (tipo 3) è stato rinominato *Inter-Area-Prefix LSA* e il tipo AS Summary Link rinominato *Inter-Area-Router LSA*.

Vengono introdotti dei nuovi tipi di LSA:

- **Grace LSA**(tipo 6): deprecato
- **NSSA LSA**(tipo 7): Permette la pubblicizzazione delle rotte esterne anche da aree stub (not-so-stubby-area)⁸
- **Link LSA**(tipo 8): lsa che contiene informazioni riguardo ai link locali
- **Intra-Area-Prefix LSA**(tipo 9) (nap lsa): lsa che danno informazioni sui prefissi dei router e link di tipo Transit Network

Gli lsa di tipo 8 e 9 permettono di recuperare le informazioni sui prefissi, in quanto verranno riportati nei payload dei suddetti. Difatto sarà possibile associare gli lsa di tipo Router e Network a tipi Link e Intra-Area-Prefix.

Scope degli LSA

Nel protocollo OSPFv3 gli LSA hanno uno specifico scope, specificato tramite lsa type. Gli scope possibili sono: link-local, area e AS. Questi possono essere sfruttati per filtrare gli lsa presenti nel database del nodo, in particolare: gli lsa con scope link-local sono del tipo link (tipo 8), mentre gli lsa con scope AS possono essere solo external (tipo 5) e infine i restanti tipi hanno tutti scope area.

⁷Nell'applicazione questo non verrà sfruttato per semplicità

⁸Non usato per semplicità

Capitolo 3

ContainerLab

ContainerLab è un software adibito allo sviluppo di topologie di rete di container. Viene, quindi, usato nell'applicazione per emulare una rete, composta di nodi e link, permettendo di configurare i container inseriti.

3.1 Vantaggi e feature

Prima di dare maggiori dettagli sul software utilizzato si vuole fornire una motivazione, per la quale il software è stato adoperato per lo sviluppo dell'applicazione. Partiamo, quindi, dai vantaggi offerti da ContainerLab:

- Viene fornita una CLI per poter organizzare le topologie di rete basate sui container, con la possibilità di gestione e di modifica
- Facilita la creazione reti di container e viene reso più facile il collegamento tra di loro usando i link
- Gran disponibilità di immagini per i container (Arista, Alpine...)

Passiamo ora ad elencare le feature offerte dal suddetto software:

- **Approccio Lab As Code(laC):** Attraverso i file di topologia di clab (file .yaml), viene fornito un metodo dichiarativo per definire i lab.
- **Documentazione:** il software dispone di una documentazione chiara e completa, per permettere l'utilizzo e la comprensione di ciò che si sta sviluppando.
- **Orchestrazione dei labs:** Per poter gestire i labs, si possono eseguire dei comandi, da terminale linux (deploy, destroy, save, inspect, graph), che dettagliano l'operazione riportando a schermo messaggi di log, con i quali è possibile ricostruire i passaggi della suddetta.

3.2 Creazione topologia di rete

Come accennato in precedenza, la topologia di rete viene creata tramite un file .clab.yaml (Yet Another Markup Language), il quale segue una determinata sintassi. Per poter comprendere meglio come strutturare questo tipo di file si riporta un esempio ¹:

¹Preso dai casi di test

```
1 name: test
2
3 topology:
4   nodes:
5     A:
6       kind: arista_ceos
7       image: ceos:4.34.0F
8       startup-config: configs/ra.cfg
9     H1:
10      kind: linux
11      image: pc_firefox_python
12      binds:
13        - /home/daniele/AutoNetMaster:/AutoNetMaster
14      ports:
15        - 5001:5000
16
17     SW:
18       kind: bridge
19
20 links:
21   - endpoints: ["A:eth4", "H1:eth1"]
```

Listing 3.1: Esempio topologia su file yaml

Il campo **name** riporta il nome del lab, questo può essere scelto dall'utente, tenendo in considerazione che verrà utilizzato per i nomi dei nodi (maggiori dettagli successivamente). La keyword **topology** è necessaria per poter iniziare a definire la topologia di rete e **qualunque** campo riguardante la suddetta dovrà essere indentato (come nodes e links), altrimenti ContainerLab restituirà un errore.

Analizziamo **nodes**, contenente le informazioni sui nodi appartenenti alla topologia di rete. Due keyword essenziali per predisporre i container sono **kind** e **image**, le quali indicano rispettivamente tipo di container e immagine utilizzata per l'emulazione. Di seguito si riportano i tre schemi possibili:

- **Router:**

1. Si indicano tipo e immagine del container
2. Viene indicato il path del file di startup-config, possibile grazie alla keyword **startup-config**²

- **Switch**

1. Necessario specificare il tipo bridge³

- **Endpoint**

1. Si indicano tipo e immagine dell'endpoint, con la differenza che quest'ultima può essere personalizzata utilizzando un dockerfile

²La ricerca dei file di configurazione partirà dalla cartella dove è presente il file yaml

³Questo tipo particolare è ricollegato ad un bridge creato nell'host

2. Per poter usare l'applicazione dovrà essere resa disponibile la directory presente nell'host, ciò è reso possibile con la keyword **binds**. Come mostrato nell'esempio sopra riportato, per poter esporre la cartella *AutoNetmaster* presente nell'host si dovrà indicare il path della suddetta e successivamente il path nel quale si vuole trovare la directory
3. Per poter accedere all'interfaccia grafica sarà necessario utilizzare il browser dell'host, dato che i container non supportano un server grafico. Quindi è necessario fare un associazione tra la porta dell'host con una porta del container⁴ e questa operazione viene effettuata con la keyword **ports**, nel quale si indicherà la porta dell'host e in seguito quella del container.

Vado a trattare ora la parte **links**, la quale costituisce tutti i collegamenti presenti tra nodi nella nostra topologia di rete. Per poter inserire un link che colleghi due endpoint, la sintassi di ContainerLab prevede di specificare, nel file `.clab.yaml` tramite la keyword **endpoints**, i due estremi del link con le rispettive interfacce utilizzate (si rimanda all'esempio per maggiore chiarezza). Si deve fare un'osservazione sulla creazione di link nella nostra topologia di rete: ContainerLab riesce ad emulare esclusivamente link con tecnologia ethernet, quindi anche nel caso in cui si vogliano collegare due dispositivi dello stesso tipo, sarà necessario ricorrere all'utilizzo di ethernet.

3.3 Deploy della topologia

Una volta che viene predisposto il file `.clab.yaml` indicante la topologia di rete, l'utente può attivare l'emulazione. Per poter operare sulla suddetta, saranno necessari comandi di ContainerLab:

Supponendo di aver creato un file `topology.clab.yaml`, per poter attivare il lab si dovrà eseguire il comando **deploy**, il quale procederà a rendere operativi i container e collegarli tra loro tramite i link dichiarati. Si fornisce un esempio di impostazione del comando:

```
sudo clab [-t topology.clab.yaml] deploy
```

Come si può evincere, il comando dovrà essere eseguito con privilegi di su e, inoltre, l'opzione `-t` non sarà necessaria qualora il deploy venga effettuato nella medesima cartella del file `yaml`. Si deve prestare attenzione qualora vengano eliminati dei container dal file `yaml`, perché potrebbe essere necessario dover aggiungere al comando l'opzione *reconfigure*, la quale non permetterà al software di considerare uno stato della topologia salvato precedentemente.

Una volta attivato il lab, sarà possibile interagire con i container tramite una CLI e per potervi accedere vi sono 2 differenti modalità:

```
ssh [nome container router/switch]@admin
```

```
docker exec -it [nome container host] bash
```

⁴porta 5000 per Flask

Il primo metodo è dedicato all'accesso di una CLI per router/switch, permettendo di eseguire i comandi di configurazione, mentre nel secondo caso verrà aperta una cli bash dell'endpoint indicato.

Nel caso in cui si voglia avere un prospetto dei container attivi, è possibile utilizzare il comando:

```
clab inspect
```

L'output sarà simile a quello riportato successivamente.

Name	Kind/Image	State	IPv4/6 Address
clab-test-A	arista_ceos ceos:4.34.0F	running	172.20.20.6 3fff:172:20:20::6
clab-test-B	arista_ceos ceos:4.34.0F	running	172.20.20.3 3fff:172:20:20::3
clab-test-C	arista_ceos ceos:4.34.0F	running	172.20.20.7 3fff:172:20:20::7
clab-test-H1	linux pc_firefox_python	running	172.20.20.5 3fff:172:20:20::5
clab-test-H2	linux pc_firefox_python	running	172.20.20.4 3fff:172:20:20::4
clab-test-H3	linux pc_firefox_python	running	172.20.20.8 3fff:172:20:20::8
clab-test-ISP	arista_ceos ceos:4.34.0F	running	172.20.20.2 3fff:172:20:20::2

Figura 3.1: Esempio di output comando clab inspect

Come si può vedere dall'immagine sopra riportata, saranno presenti i nomi dei container e altre informazioni su di essi. Nel caso in cui si desideri interrompere l'emulazione della nostra topologia, si potrà eseguire il comando:

```
sudo clab destroy
```

Tale comando disattiverà tutti i container, quindi interrompendo l'emulazione della topologia di rete.

Capitolo 4

Back-end in python

L'applicazione è stata realizzata utilizzando il linguaggio di programmazione python, che ha permesso di poter sfruttare i moduli **pyeapi** e **Flask**. Il programma prevede una parte in back-end, con la quale si possono recuperare le informazioni necessarie dalla topologia di rete, emulata con ContainerLab, e una parte in front-end, con la quale è possibile effettuare una rappresentazione grafica dello schema di rete grazie al servizio di hosting offerto da Flask e la libreria Vis.js. Per trattare i contenuti dell'applicazione si suddivide l'analisi in parte di back-end e la parte in front-end.

In questa sezione andremo quindi ad analizzare come l'applicazione recupera e gestisce le informazioni della topologia di rete emulata. Un **passaggio preliminare necessario** è quello di **emulare la topologia** sfruttando ContainerLab, altrimenti l'applicazione restituisce un errore di Timeout, il motivo verrà spiegato successivamente. Inoltre un altro **requisito** è quello di aver installato il modulo **pyeapi**, perché le funzioni, offerte dal suddetto modulo, sono essenziali per il recupero dei dati dalla topologia di rete emulata. In particolare andiamo a trattare delle funzioni usate per poter sviluppare l'applicazione. Il modulo pyeapi, realizzato da Arista, è una libreria wrapper focalizzata sull'utilizzo di Arista EOS eAPI, permettendo di stabilire connessioni con i container e inviare comandi. Gli output dei comandi, lanciati sui nodi, vengono restituiti in un formato json, con l'ausilio del protocollo JSON-RPC.

4.1 Pyeapi client

Il modulo pyeapi è stato sviluppato con lo scopo di poter comunicare , attraverso delle funzioni in python, con i nodi emulati utilizzando un'immagine Arista. Le funzioni utilizzate in questa applicazione sono 2: **connect** e **enable**.

4.1.1 Connect

La funzione connect permette di stabilire una connessione con un nodo Arista EOS.

```
1 import pyeapi
2
3 nodo=pyeapi.client.connect(
4     transport='https',
5     host='10.0.0.1',
6     username='admin',
```

```
7     password='admin',
8     return_node=True
9 )
```

Listing 4.1: Esempio funzione Connect

Come si può vedere dall'esempio riportato sopra, la funzione connect necessita di qualche parametro, si riportano solo quelli essenziali, mentre gli altri assumeranno i valori di default prestabiliti.

- il parametro **transport** definisce il protocollo utilizzato per lo scambio dei messaggi con il nodo ed è stato impostato ad *https*
- il parametro **host** contiene l'indirizzo del nodo con cui stabilire una connessione¹
- **username** e **password** sono le credenziali dell'utente amministratore del nodo (dipendono dalla configurazione del nodo genericamente)
- **return node** settato a True, perché si vuole riutilizzare l'istanza del nodo

4.1.2 Enable

La funzione enable invia il comando specificato al nodo Arista EOS e restituisce la risposta in formato json.

```
1 import pyeapi
2
3 nodo=pyeapi.client.connect(
4     transport='https',
5     host='10.0.0.1',
6     username='admin',
7     password='admin',
8     return_node=True
9 )
10
11 result=nodo.enable('ping 10.0.0.2')
```

Da come si può vedere nell'esempio, il comando viene passato sotto forma di stringa, mentre il valore ritornato dalla funzione sarà un elemento strutturato in json (questo aspetto può essere cambiato assegnando un valore al parametro *encoding*). Se si vogliono maggiori informazioni sul suddetto risultato, si può aprire il browser e digitare nella barra dell'URL *[indirizzo interfaccia nodo]/eapi*, così facendo si aprirà una pagina web in grado di inviare comandi al nodo e mostrare le response in json.²

¹Nell'applicazione verrà usato l'indirizzo dell'interfaccia di loopback

²Strumento citato perché utilizzato in fase di sviluppo

4.2 Classi

Come già anticipato, l'applicazione recupererà i dati dalla topologia di rete emulata e dovrà gestirli, anche in previsione di mostrarli all'utente. Per poter permettere ciò, vengono definite delle classi (nel file *utilities.py*) che vengono impiegate per la memorizzazione e formattazione dei dati suddetti. L'analisi verrà effettuata partendo dalle classi che non hanno bisogno di un'altra per essere definite. Prima di iniziare è opportuno specificare che, per permettere la formattazione di queste informazioni, ogni classe dispone di un metodo *toJSON()*, che costruisce un elemento JSON partendo dagli attributi del chiamante. In alcune classi, per le quali è richiesta una stampa a video, viene anche definito il metodo *__str__*, che permette di costruire una stringa con il riepilogo di tutti i dati dell'istanza. I suddetti metodi non verranno mostrati nei blocchi di codice sottostanti, perché si vuole riportare solo il contenuto essenziale della classi³.

4.2.1 Path_To_ASBR

Questa classe è dedicata a memorizzare le informazioni di una rotta verso ASBR, la quale verrà pubblicizzata all'interno dell'AS. Le informazioni memorizzate sono le seguenti: Router ID del ASBR, l'indirizzo del next hop per raggiungere ASBR e la metrica associata a questa rotta.

```
1 class Path_To_ASBR:
2     def __init__(self, asbr, via, metric):
3         self.asbr = asbr
4         self.via = via
5         self.metric = metric
```

Listing 4.2: Classe per gestire rotte verso ASBR

4.2.2 Route

La classe Route permette di gestire le informazioni di una rotta presente nella tabella di routing di un nodo. La classe è stata modificata, rispetto alla versione per rotte IPv4, per poterla adeguare alle informazioni date dalla tabella di routing IPv6, in quanto si potrebbe riscontrare più di un next hop disponibile per una rotta di routing.

```
1 class Route:
2     def __init__(self, ip, masklen, via, metric, metric_type=None):
3         self.ip = ip
4         self.masklen = masklen
5         self.via = [via]
6         self.metric = metric
7         self.metric_type = metric_type
8
```

³Si rimanda alla lettura del file di codice utilities.py

```
9     def add_via(self, other_via):
10         self.via.append(other_via)
```

Listing 4.3: Classe per gestione delle rotte IPv6

Nella classe viene definito anche un metodo *add_via*, che permette di supportare l'aspetto indicato precedentemente.

4.2.3 Link

Per poter gestire i link presenti nella topologia di rete, viene predisposta la classe Link.

```
1 class Link:
2     def __init__(self, id, type, options, metric, prefix=None,
3                 endpoints=None, dr=None, bdr=None):
4         self.id = id
5         self.type = type
6         self.options = options
7         self.metric = metric
8         self.endpoints = endpoints if endpoints else []
9         self.prefix = prefix
10        self.dr = dr
11        self.bdr = bdr
12
13    def add_endpoint(self, endpoint):
14        if not(endpoint in self.endpoints):
15            self.endpoints.append(endpoint)
16
17    def set_dr_bdr(self, dr, bdr):
18        self.dr = dr
19        self.bdr = bdr
```

Listing 4.4: Classe per gestire i link della topologia

Vi è bisogno di dettagliare gli attributi e gli ulteriori metodi forniti dalla classe. Partendo dai primi:

- possiamo dire **prefix** contiene indirizzo di rete e lunghezza del prefisso associate al link
- **id** contiene esclusivamente il prefisso del link (senza lunghezza) e viene utilizzato per identificare il link
- **type**, **options** e **metric** riguardano le caratteristiche fisiche associate al link, infatti avremo il tipo di interfaccia, le informazioni aggiuntive sul link specificate tramite options e la metrica associata ad esso.
- **br** e **bdr** sono rispettivamente i router id di DR (Designated Router) e BDR (Backup Designated Router), attributi necessari nel caso in cui il tipo di link sia *Transit network*, altrimenti i campi non sono significativi.

- **endpoints** contiene i router id di tutti i nodi che sono collegati a quel link direttamente o tramite switch

Dopo aver analizzato gli attributi, passiamo a trattare i metodi definiti:

- **add_endpoint** : metodo fornito per poter inserire il router id di un endpoint (passato come parametro) nella lista di endpoint collegati al nodo.
- **set_br_dbr** : Come intuibile dal nome stesso del metodo, si vanno ad impostare i router id, passati come parametro, del DR e BDR rispettivamente.

4.2.4 Node

La classe Node contiene tutte le informazioni necessarie e utili per un nodo che opera al livello di rete nella pila protocollare. Viene mostrata quindi la suddetta classe:

```

1 class Node:
2     def __init__(self, router_id, hostname, interface_list=None,
3                 neighbor_list=None, route_table=None):
4         self.hostname = hostname
5         self.router_id = router_id
6         self.interfaces = interface_list if interface_list else
7         []
8         self.neighbors = neighbor_list if neighbor_list else []
9         self.route_table = route_table if route_table else []
10
11     def add_interface(self, id, add, interface_status,
12 line_protocol_status):
13         self.interfaces.append({
14             "id": id,
15             "addresses": add,
16             "interface_status": interface_status,
17             "line_protocol_status": line_protocol_status
18         })
19
20     def add_neighbor(self, interface_id, neighbor_router_id,
21 neighbor_ip_addr,
22 adjacency_state, designated_router,
23 backup_designated_router):
24         self.neighbors.append({
25             "interface_id": interface_id,
26             "router_id": neighbor_router_id,
27             "adjacency_state": adjacency_state,
28             "designated_router": designated_router,
29             "backup_designated_router": backup_designated_router
30         })
31
32     def add_route(self, ip, masklen, vias, protocol):
33         self.route_table.append({
34             "ip": ip,
35             "prefixLen": masklen,

```

```

32         "vias":vias ,
33         "protocol": protocol
34     })

```

Listing 4.5: Classe per la gestione dei nodi

Per iniziare l'analisi di questa classe, parliamo degli attributi: **hostname** e **router_id** contengono rispettivamente hostname e router id del nodo; **interfaces**, **neighbors** e **route_table** contengono in ordine: interfacce del nodo⁴, vicini OSPFv3 del nodo e route table IPv6. I metodi forniti supportano l'inserimento nelle liste disposte dalla classe, in particolare, **add_interface** verrà utilizzato per inserire un'interfaccia alla lista *interfaces* e, in maniera analoga, avverrà con **add_neighbor** nella lista *neighbors* e con **add_route** in *route_table*. Si aggiunge, a ciò che è stato detto precedentemente, che la classe presenta delle modifiche nella logica (rispetto alla versione per OSPFv2): nella lista *neighbors* non sono presenti indirizzi IPv6 dei nodi, il perché dietro questo cambiamento risiede nel fatto che l'output del comando dedicato (*show ipv6 ospf neighbors*) non riporta alcuna informazione sugli indirizzi IP, in quanto è necessario solo il router id. Il secondo cambiamento invece si presenta nelle interfacce, in particolare, si ricorda che IPv6 ammette la configurazione di più di un indirizzo in un'interfaccia (più dettagli al paragrafo 2.2.1), portando quindi a strutturare l'attributo *addresses* come lista.

4.2.5 Area

Adesso che sono state spiegate le classi per definire i link, i nodi e le rotte, si può introdurre la classe per gestire le informazioni su un'area dell'AS.

```

1 class Area:
2     def __init__(self, area_id):
3         self.area_id = area_id
4         self.nodes = set()
5         self.links = set()
6         self.ospf_inter_area_routes = set()
7         self.paths_to_asbrs = set()
8
9     def add_node(self, node):
10        self.nodes.add(node)
11
12    def add_link(self, link):
13        self.links.add(link)
14
15    def add_inter_area_route(self, route):
16        self.ospf_inter_area_routes.add(route)
17
18    def add_path_to_asbr(self, path):
19        self.paths_to_asbrs.add(path)
20
21    def search_route(self, ip, masklen):

```

⁴L'output terrà in considerazione esclusivamente la configurazione IPv6

```
22         for r in self.ospf_inter_area_routes:
23             if r.ip==ip and r.masklen==masklen:
24                 return r
25         return None
```

Listing 4.6: Classe per la gestione delle aree

Come si può osservare dal codice riportato sopra, un'istanza dell'area OSPF è composta da nodi (attributo *nodes*), link (attributo *links*), rotta inter-area (attributo *ospf_inter_area_routes*) e di percorsi verso il router ASBR (attributo *paths_to_asbrs*), qualora il router ASBR non fosse presente nell'area. Sono stati introdotti anche i metodi per aggiungere un elemento alle suddette liste e un metodo per cercare una specifica rotta con parametri un determinato indirizzo con un ip e maschera. Quest'ultima viene utilizzata quando si recuperano le informazioni sulle rotte già esistenti, in modo tale da non duplicare le rotte con medesima destinazione.

4.2.6 Network

Ora che sono state espone tutte le classi necessarie, si può analizzare la classe che le riunisce tutte, la classe *Network*, che modella la topologia, quindi formata da diverse aree e rotte verso le reti esterne alla rete dell'organizzazione.

```
1 class Network:
2     def __init__(self):
3         self.areas = set()
4         self.external_routes = set()
5
6     def add_area(self, area):
7         self.areas.add(area)
8
9     def add_external_network(self, route):
10        self.external_routes.add(route)
11
12    def find_target_area(self, area_data):
13        target_area = None
14        for area in self.areas:
15            if area.area_id == area_data:
16                target_area = area
17                break
18        return target_area
```

Listing 4.7: Classe per la gestione della rete

Le liste **areas** e **external_routes** contengono rispettivamente istanze della classe *Area* e *Route* (per le rotte esterne). Inoltre è stato implementato il metodo *find_target_area*, che fornisce l'istanza della classe *Area*, contenuta nella lista *areas*, con id uguale al parametro *area_data* passato. Questo metodo verrà utilizzato quando si dovrà recuperare l'istanza di un'area partendo dal proprio id, il quale verrà recuperato nel momento in cui gli lsa saranno processati.

4.3 Funzioni getter lsa

Per poter prelevare gli lsa, l'applicazione prevede dei moduli dedicati, uno per ogni tipo di lsa, dove in ognuno è stato implementato un metodo che preleva gli lsa e li utilizza per costruire un elemento JSON. Si deve specificare che, con l'utilizzo di OSPFv3, per poter ottenere gli lsa di un determinato tipo, è necessario specificare anche l'area in cui vengono annunciati, questo comporta una divisione degli lsa per aree. Questa caratteristica viene mantenuta nell'elemento JSON costruito, difatti verrà fatta una lista di lsa e associata all'area id corrispondente. Queste operazioni vengono effettuate nel file *router_lsa.py*, *network_lsa.py* ecc. Si riporta un esempio del codice sopra menzionato:

```

1 import pyeapi
2 import json
3 from areas.get_areas import get_areas
4 def get_router_lsa_info(target_node):
5     "Funzione che recupera gli lsa di tipo router dal database
6     OSPFv3 del nodo"
7
8     command= "show ipv6 ospf database area "
9
10    node_areas= get_areas(target_node)
11    lsa_type_1={}
12    # dato che non esiste un comando che da tutti gli lsa di tipo
13    # router senza specificare l'area
14    # bisogna eseguire il comando per ogni area
15    for a in node_areas:
16        output=target_node.enable(command+a+" router detail")
17        area_entries= output[0]["result"]["vrfs"]["default"]["
18        instList"]["10"]["ospf3AreaEntries"]
19        if not(area_entries[str(a)]["ospf3AreaLsaList"]==[]):
20            lsa_type_1[str(a)]=area_entries[str(a)]
21
22    return lsa_type_1

```

Listing 4.8: Codice per prelevare router lsa delle aree

Come si può evincere dal codice riportato sopra, verrà costruito un **dictionary**⁵, formato nel seguente modo: per ogni area si avrà un elemento del tipo `area_id : lista_area_lsa`. Questa struttura risulta vantaggiosa, perché è facilmente iterabile, con i costrutti forniti da python, e risulta anche efficace nel riportare le informazioni in maniera semplice e precisa. Unica eccezione viene fatta dai link lsa e gli external lsa, perché gli elementi JSON in questo caso vengono già predisposti per essere processati. Per una migliore chiarezza si riportano i blocchi di codice per la gestione dei link lsa e external lsa:

```

1 import pyeapi
2 import json
3

```

⁵struttura dati usata in python

```

4 def get_external_lsa_info(target_node):
5     "Funzione che recupera gli lsa di tipo as external dal
      database OSPFv3 del nodo"
6
7     command = "show ipv6 ospf database as detail"
8
9     output = target_node.enable(command)
10
11     lsa_5_list = output[0]['result']['vrfs']['default']['instList
      ']['10']['ospf3AsLsas']
12
13     return lsa_5_list

```

Listing 4.9: Codice per il recupero degli external lsa

```

1 import pyeapi
2 import json
3 def get_link_lsa_info(target_node):
4     "Funzione che recupera gli lsa di tipo link dal database
      OSPFv3 del nodo"
5
6     command= "show ipv6 ospf database link detail "
7
8     output= target_node.enable(command)
9
10     lsa_type_8= output[0]["result"]["vrfs"]["default"]["instList"
      "]["10"]["ospf3InterfaceEntries"]
11
12     return lsa_type_8

```

Listing 4.10: Codice per il recupero dei link lsa

4.4 File main

Nel file *main.py* risiede il core dell'applicazione sviluppata, perché riunisce tutte le componenti descritte precedentemente e le utilizza per raccogliere tutte le informazioni sulla topologia di rete, predisponendole poi per l'interfaccia grafica web-based. Iniziamo quindi la trattazione esplicitando che il codice opera in due fasi: il recupero delle informazioni e la fase in cui è in attesa di comandi dell'utente. Quindi verrà introdotta prima la fase del recupero delle informazioni, in quanto è la fase chiave per ricostruire la topologia, e, successivamente, la fase in cui il programma provvederà a mostrare gli output dei comandi, i quali opereranno sulle informazioni della topologia precedentemente ricostruita.

4.4.1 Dettagli sul nodo scelto

L'applicazione prevede il passaggio di un parametro, il quale rappresenta l'indirizzo IPv4 dell'interfaccia di loopback del nodo scelto, dal quale iniziare a ricostruire la topologia di rete.

Una volta instaurata la connessione, tramite il metodo *connect* di *pyeapi*, si recupereranno le seguenti informazioni: hostname, configurazione interfacce IPv6, route table IPv6, informazioni sul protocollo OSPFv3 e i vicini OSPFv3. In seguito, viene effettuata una stampa di tutte le informazioni del nodo, resa possibile grazie al metodo *__str__* della classe *Node*, in modo tale da fornire all'utente una prospettiva sulla configurazione del router scelto.

```

1 if len(sys.argv) < 2:
2     sys.stderr.write('ERRORE: nodo target non fornito in input\n'
3     )
4     sys.exit(1)
5 input_node = sys.argv[1]
6
7 target_node = pyeapi.client.connect(
8     transport='https',
9     host=input_node,
10    username='admin',
11    password='admin',
12    return_node=True
13 )
14
15 hostname = get_hostname(target_node)
16
17 interfaces = get_interfaces(target_node)
18
19 route_table = get_route_table(target_node)
20
21 protocol_info = get_protocol_info(target_node)
22
23 neighbors = get_neighbors(target_node)
24
25 router = Node(protocol_info['Router ID'], hostname, interfaces,
26               neighbors, route_table)
27
28 print(f"\n{router}\n")

```

Listing 4.11: Recupero informazioni iniziali nodo

4.4.2 Ricostruzione della topologia

Proseguendo nella trattazione, il programma provvede a creare un'istanza della classe **Network**, i cui attributi, con il recupero dei vari tipi lsa, verranno modificati. Il punto iniziale della ricostruzione sono i **router_lsa**, i quali forniscono informazioni sui link connessi al nodo scelto. Dato che i router lsa non contengono informazioni sui prefissi dei link, sarà necessario utilizzare anche i link lsa.

```

1 network_topology = Network()
2 router_lsa_1 = get_router_lsa_info(target_node)

```



```

3 link_lsa_8 = get_link_lsa_info(target_node)
4 nap_lsa_9= get_nap_lsa_info(target_node)
5
6 for area_data in router_lsa_1:
7     new_area = Area(area_data)
8     for lsa_entry in router_lsa_1[area_data]['ospf3AreaLsaList']:
9         link_state_id = lsa_entry['linkStateId']
10        advertising_router = lsa_entry['advertisingRouter']
11        # devo cambiare con Advertising router perche' il campo
12        # link state id non da piu' info sul router ID in OSPFv3
13        new_area.add_node(advertising_router)
14
15        for router_link in lsa_entry['ospf3RouterLsa']['
routerLsaLinks']:
16            interface_id = router_link['interfaceId']
17            interface_type = router_link['interfaceType']
18            neighbor_router_id= router_link['neighborRouterId']
19            neighbor_interface_id=router_link['
neighborInterfaceId']
20            metric = router_link['metric']
21
22            #-----#
23            # codice listing 4.13 #
24            #-----#
25
26
27            # si tratta di un link di cui non posso recuperare le
informazioni direttamente
28            # ma con l'aiuto di un altro nodo
29            if link_id=="":
30                continue
31
32
33            existing_link = None
34            for link in new_area.links:
35                if link.id == link_id
36                   and link.type == interface_type:
37                    existing_link = link
38                    break
39
40            # dato che in OSPFv3 non sono contenuti indirizzi
negli header
41            # invece di inserire linkStateID inserisco
AdvertisingRouter
42
43            if existing_link:
44                existing_link.add_endpoint(advertising_router)
45            else:
46                ins_endpoints=[]
47                if(advertising_router != neighbor_router_id):
48                    ins_endpoints.append(neighbor_router_id)
49                    ins_endpoints.append(advertising_router)

```

```

49         new_link = Link(id=link_id, type=interface_type,
options=None, metric=metric, endpoints=ins_endpoints, prefix=f
"{link_id}/{prefix_l}")
50         new_area.add_link(new_link)
51
52     network_topology.add_area(new_area)

```

Listing 4.12: Codice di gestione per i router lsa

Per poter mostrare il codice in maniera più ordinata possibile, il codice che fa uso dei link lsa viene riportato nel listing successivo.

```

1 link_id=""
2 prefix_l=0
3 # per trovare il link id devo andare a trovare lsa di tipo 8 che
   contiene questa informazione
4 # devo trovare coppia di lsa 8 con questi router id e interface
   id
5 for ll in link_lsa_8:
6     for lsa8_entry in link_lsa_8[ll]['ospf3InterfaceLsaList']:
7         if len(lsa8_entry['ospf3LinkLsa']['prefixList']) == 0:
8             continue
9         if lsa8_entry['linkStateId'] == interface_id
10            and lsa8_entry['advertisingRouter'] ==
advertising_router:
11             l8_e=lsa8_entry['ospf3LinkLsa']['prefixList'][0]
12             # riguarda il router advertising
13             prefix_l=l8_e['prefixLength']
14             link_id=l8_e['prefix']
15             break
16         elif lsa8_entry['linkStateId'] == neighbor_interface_id
17            and lsa8_entry['advertisingRouter'] ==
neighbor_router_id:
18             # ho informazioni sul vicino
19             prefix_l=l8_e['prefixLength']
20             link_id=l8_e['prefix']
21             break

```

Listing 4.13: Codice necessario per link lsa

Come si può notare dai blocchi precedenti, avendo a disposizione i router lsa si possono scoprire le aree alle quali il nodo fa parte e i link appartenenti ad una determinata area. Ricorrere quindi solo alle informazioni dei router lsa non è sufficiente, perché sono necessarie anche le informazioni sui prefissi, reperibili tramite i link lsa. Per poter associare correttamente le informazioni date dai link lsa ad un determinato router lsa, ci sarà il bisogno di controllare degli specifici campi: **linkStateId** e **advertisingRouter**. Questi forniscono il prefisso e la sua lunghezza da poter assegnare all'istanza Link, perché informazioni associate sia al nodo stesso, sia ad un vicino⁶. Una volta acquisiti prefisso e la relativa lunghezza, si potranno inserire

⁶Si è deciso di fare controllo dei vicini anche per reperire l'informazione dal vicino

gli endpoint e infine il link nell'area. Questi step vengono ripetuti per ogni router lsa associato ad una determinata area, quindi questo comporta che questi passaggi debbano essere svolti per ogni area e, una volta completata la scoperta della nostra area, questa viene aggiunta alla topologia creata. Si deve menzionare un dettaglio, tutti gli step indicati precedentemente non permettono di trovare le stub network, per farlo dovremo ricorrere agli intra-area-prefix lsa, in quanto conterranno le informazioni sui prefissi delle stub Network e le associeranno a diversi router lsa, viene riportato il codice di seguito:

```

1 #recupero le informazioni sulle stub network dagli nap lsa
2 for ll in nap_lsa_9:
3     for lsa_9_entry in nap_lsa_9[ll]['ospf3AreaLsaList']:
4         lsa_nap=lsa_9_entry['ospf3IntraAreaPrefixLsa']
5         if lsa_nap['referencedLsaType']!='routerLsa':
6             continue
7         link_id=lsa_nap['prefixList'][0]['prefix']
8         prefix_l=lsa_nap['prefixList'][0]['prefixLength']
9         metric=lsa_nap['prefixList'][0]['metric']
10        new_link = Link(id=link_id, type="stubNetwork", options=
None, metric=metric, endpoints=[lsa_nap['
referencedAdvertisingRouter']] ,prefix=f"{link_id}/{prefix_l}"
11        )
        network_topology.find_target_area(ll).add_link(new_link)

```

Listing 4.14: Codice per il recupero delle stubNetwork

Passiamo ad analizzare la gestione che effettua il programma per i **network LSA**. Essi forniscono le informazioni riguardanti il DR e il BDR, che gestiranno la rete ad accesso multiplo, alla quale sono connessi i router specificati, nell'oggetto json, tramite router id nel valore con chiave **Attached Routers**. Anche in questo caso, sfruttare i dati ottenuti dal suddetto tipo di lsa non forniranno tutte le informazioni necessarie, perciò sarà richiesto l'ausilio degli intra-area-prefix lsa, che, oltre ad aggiungere informazioni sui router lsa, possono essere associati a network lsa. Come nel caso dei router lsa, dovranno essere verificate delle condizioni sul prefisso associato al link, però è importante evidenziare che i nap lsa forniscono informazioni anche su link non adiacenti al nodo, che devono essere memorizzate se si vuole ricostruire completamente la topologia.

```

1 # recupero LSA tipo 2
2 network_lsa_2 = get_network_lsa_info(target_node)
3
4 for area_data in network_lsa_2:
5     for lsa_entry in network_lsa_2[area_data]['ospf3AreaLsaList'
]:
6         link_state_id = lsa_entry['linkStateId']
7         dr = lsa_entry['advertisingRouter']
8         attached_routers = lsa_entry['ospf3NetworkLsa']['
attachedRouters']
9         #cambiare la scelta del DBR, potrei rischiare di
inserire il DR anche come BDR
10        bdr = attached_routers[0] if len(attached_routers) >

```

```

1  else None
11
12      #-----#
13      #     codice listing 4.16     #
14      #-----#

```

Listing 4.15: Codice per gestione dei network lsa

```

1
2  # devo scorrere i nap lsa per poter ottenere informazione
3  # su network lsa
4  adjacent=True
5  network_prefix=""
6  for lsa9_entry in nap_lsa_9[area_data]['ospf3AreaLsaList']:
7      l9_e=lsa9_entry['ospf3IntraAreaPrefixLsa']
8  if l9_e['referencedLsaType']!='networkLsa':
9      continue
10 elif protocol_info['Router ID'] in attached_routers
11     and dr == l9_e['referencedAdvertisingRouter']
12     and link_state_id == l9_e['referencedLinkStateId']:
13     network_prefix= l9_e['prefixList'][0]['prefix']
14 elif protocol_info['Router ID'] not in attached_routers:
15     #link non adiacente
16     #prendo il dr e trovo il nap annunciato
17     if dr == l9_e['referencedAdvertisingRouter']
18         and link_state_id == l9_e['referencedLinkStateId']:
19         #prendo prefix
20         network_prefix= l9_e['prefixList'][0]['prefix']
21         link_id=l9_e['prefixList'][0]['prefix']
22         prefix_l=l9_e['prefixList'][0]['prefixLength']
23         metric=l9_e['prefixList'][0]['metric']
24         new_link = Link(id=link_id, type="transitNetwork",
options=None, metric=metric, endpoints=attached_routers ,
prefix=f"{link_id}/{prefix_l}")
25         new_link.set_dr_bdr(dr=dr, bdr=bdr)
26         network_topology.find_target_area(area_data).add_link
(new_link)
27         adjacent=False
28
29 # si tratta di un link adiacente e di cui ho il prefisso
30 if adjacent:
31     for link in network_topology.find_target_area(area_data=
area_data).links:
32         # qua devo fare il confronto con prefix
33         if link.id == network_prefix:
34             link.set_dr_bdr(dr, bdr)

```

Listing 4.16: Utilizzo nap lsa per network lsa

A questo punto la scoperta di ogni informazione interna ad ogni area è completa, rimane, quindi, da completare la scoperta della topologia definendo le rotte inter-area, i percorsi verso ASBR e le rotte esterne. Per ottenere le suddette informazioni si dovrà ricorrere ai seguenti tipi di lsa: inter-area-prefix, inter-area-router e external.

Le aree riescono a comunicare tra loro grazie alle rotte inter-area, queste possono essere pubblicizzate nella rete dell'AS tramite gli **inter-area-prefix lsa**. Viene fatta inoltre un'ottimizzazione dovuta al fatto che, nella tabella di routing IPv6, una destinazione potrebbe avere più next hop possibili per essere raggiunta, quindi l'algoritmo non crea doppia rotta con medesima destinazione, bensì aggiunge una via ad una eventuale rotta già esistente.

```

1 # recupero LSA tipo 3
2
3 iar_lsa_3 = get_iar_lsa_info(target_node)
4
5 for area_data in iar_lsa_3:
6     target_area = network_topology.find_target_area(area_data)
7     if not target_area:
8         continue
9
10    for lsa_entry in iar_lsa_3[area_data]['ospf3AreaLsaList']:
11        l3_e=lsa_entry['ospf3InterAreaPrefixLsa']
12        ip = l3_e['prefix']['prefix']
13        mask = l3_e['prefix']['prefixLength']
14        via = lsa_entry['advertisingRouter']
15        metric = l3_e['metric']
16
17        existing=target_area.search_route(ip,mask)
18        if existing==None:
19            route = Route(ip, mask, via, metric)
20
21            target_area.add_inter_area_route(route)
22        else:
23            existing.add_via(via)

```

Listing 4.17: Gestione intra-area-prefix lsa

Si passa ora alla gestione degli **inter-area-router lsa**, i quali permettono la pubblicizzazione di rotte per raggiungere ASBR. Queste sono fondamentali per poter raggiungere l'esterno dell'AS e dovranno essere distribuite in tutte le aree.

```

1 # recupero LSA tipo 4
2
3 iar_lsa_4 = get_iar_lsa_info(target_node)
4
5 for area_data in iar_lsa_4:
6     target_area = network_topology.find_target_area(area_data)
7     if not target_area:
8         continue
9

```

```

10     for lsa_entry in iar_lsa_4[area_data]['ospf3AreaLsaList']:
11         l4_e=lsa_entry['ospf3InterAreaRouterLsa']
12         asbr = l4_e['destinationRouterId']
13         via = lsa_entry['advertisingRouter']
14         metric = l4_e['metric']
15
16         path = Path_To_ASBR(asbr, via, metric)
17
18         target_area.add_path_to_asbr(path)

```

Listing 4.18: Gestione intra-area-router lsa

A questo punto non resta che analizzare il codice predisposto per la gestione degli **external lsa**, annunciati da ASBR e che permettono alla rete interna di conoscere destinazioni al di fuori della rete dell'AS. A differenza delle rotte precedentemente considerate, sarà necessario riportare anche il campo `metricType`, in quanto informazione presente per le rotte esterne.

```

1  # recupero LSA di tipo 5
2
3  external_lsa_5 = get_external_lsa_info(target_node)
4
5  for lsa in external_lsa_5:
6      l5_e=lsa['ospf3ExternalLsa']
7      ip = l5_e['prefix']['prefix']
8      prefix_len = l5_e['prefix']['prefixLength']
9      via = lsa['advertisingRouter']
10     metric = l5_e['metric']
11     metric_type = l5_e['metricType']
12
13     route = Route(ip, prefix_len, via, metric, metric_type)
14
15     network_topology.add_external_network(route)

```

Listing 4.19: Gestione external lsa

Nei casi reali le operazioni svolte fino ad ora sarebbero insufficienti per la scoperta della topologia di rete e il motivo risiede nel fatto che, in una rete reale, un qualsiasi nodo non potrà risultare vicino di tutti gli altri. Per superare questo ostacolo, è stato implementato il seguente algoritmo, con lo scopo di sfruttare le informazioni dei nodi vicini per poter completare la topologia della rete. Questa soluzione risulta efficace, in quanto si sfrutta l'informazione dei vicini OSPFv3, la quale permette, oltre che acquisire ulteriori dettagli su questi, di scoprire anche i nodi non vicini al nodo scelto all'avvio dell'applicazione. Per poter implementare questo algoritmo, ci si appoggia ad una queue, che viene riempita con delle istanze `Node` dei nuovi nodi scoperti, da cui si andranno a trovare altri vicini. Allo scopo di ottenere le informazioni sui nuovi nodi non vicini, si fa uso della funzione `connect` di `pyeapi`, che sfrutta il router id (ip delle interfacce di loopback), per effettuare la connessione ai suddetti nodi e recuperare le seguenti informazioni: interfacce configurate con IPv6, route table IPv6, dati sul protocollo OSPFv3 e vicini OSPFv3.

```

1 # discover other nodes
2 def discover_router(ip_addr):
3     """Connette al router dato l'IP e ne estrae le informazioni."""
4     node = pyeapi.client.connect(
5         transport='https',
6         host=ip_addr,
7         username='admin',
8         password='admin',
9         return_node=True
10    )
11
12    hostname = (node.enable('show hostname'))[0]['result']['hostname']
13    interfaces = get_interfaces(node)
14    route_table = get_route_table(node)
15    protocol_info = get_protocol_info(node)
16    neighbors = get_neighbors(node)
17
18    router_id = protocol_info['Router ID']
19
20    return Node(router_id, hostname, interfaces, neighbors,
21               route_table), neighbors
22
23 # Mappa router_id -> Node
24 network_routers = {router.router_id: router}
25 # Mappa router_id -> IP dei vicini
26 discovered_ips = {router.router_id: router.neighbors}
27 queue = Queue()
28 queue.put(router)
29
30 while not queue.empty():
31     current_router = queue.get()
32     neighbors_dict = {n["router_id"]: n for n in current_router.neighbors}
33
34     # utilizzo il router ID perche' sara' l'indirizzo dell'
35     # interfaccia di loopback
36     # del vicino a cui posso collegarmi per recuperare le
37     # informazioni
38     for nghb in current_router.neighbors:
39         nghb_id = nghb['router_id']
40
41         if nghb_id not in network_routers: # and nghb_id !=
42             router.router_id:
43                 new_router, new_neighbors = discover_router(nghb_id)
44                 network_routers[nghb_id] = new_router
45                 queue.put(new_router)

```

Listing 4.20: Codice di scoperta nodi non vicini

Ultimati tutti i preparativi per avere completa conoscenza della topologia, si può passare a trattare la parte del file main che si occupa di "eseguire" i comandi richiesti dall'utente. Il tutto viene reso possibile dal codice riportato in seguito, che fa uso dei dati raccolti con l'algoritmo descritto precedentemente ed eventualmente predisponendo il necessario per l'interfaccia grafica web-based (maggiori dettagli in seguito). L'utente potrà eseguire differenti comandi, che potrà visualizzare inviando il comando *help*, si offre comunque una breve descrizione di questi:

- **id [ospf_id]:** permette di stampare a video le informazioni del nodo con ospf router id passato
- **nodes:** permette di stampare tutte le informazioni sui nodi presenti nella topologia di rete
- **topology:** effettua la stampa a video di tutte le informazioni ottenute per scoprire la topologia di rete
- **display:** attiva il servizio di hosting Flask per la gui web-based

```

1 while True:
2     cmd = input("> ").strip()
3
4     if cmd.startswith("id "):
5         id = cmd[3:]
6         matching_router = network_routers.get(id)
7         if(matching_router != None):
8             print(matching_router)
9             print('\n')
10        else:
11            print("Non esistono router con l'id selezionato")
12
13        elif cmd == "nodes":
14            print('NODES:\n')
15            for router in network_routers.values():
16                print(router)
17                print('\n')
18
19        elif cmd == "topology":
20            print(network_topology)
21
22        elif cmd == "display":
23            network_routers_json = {router_id: json.loads(router.
24            toJSON()) for router_id, router in network_routers.items()}
25            data = network_topology.toJSON()
26
27            app.config['NETWORK_ROUTERS_JSON'] =
28            network_routers_json
29            app.config['DATA'] = data
30            app.config['TARGET'] = input_node
31
32            flask_thread = Thread(target=run_flask, daemon=True)

```



```
31         flask_thread.start()
32
33     elif cmd == "help":
34         print("""
35         Comandi disponibili:
36             id [ospf_id]          - Ottieni informazioni sul nodo di
rete l'ID ospf specificato.
37             nodes                  - Mostra le caratteristiche dei
nodi della rete
38             topology              - Stampa la topologia della rete.
39             display                - Avvia un'interfaccia web per la
visualizzazione grafica della topologia.
40             exit                  - Esci dal programma.
41         """)
42
43     elif cmd == "exit":
44         sys.exit(0)
45
46     else:
47         print("Comando non riconosciuto. Digita 'help' per
assistenza.")
```

Listing 4.21: Gestione intra-area-prefix lsa

Capitolo 5

Front-end web-based

Il seguente capitolo tratterà della parte front-end, sviluppata nell'applicazione, e sul relativo utilizzo. Prima di iniziare si deve specificare che tale interfaccia grafica è pensata per essere web-based, cioè che per poter essere utilizzata si ricorre all'uso di un browser web, con l'ausilio di altre risorse adibite a creare la pagina web. Inoltre, dato che questa applicazione è frutto di un'estensione, le modifiche effettuate sull'interfaccia grafica sono minime, per questa ragione verranno indicati esclusivamente i cambiamenti¹. Con lo scopo di rendere chiara la spiegazione della gui, si introducono prima le componenti utilizzate per lo sviluppo.

5.1 Hosting Flask

Una risorsa necessaria, per offrire all'utente una pagina web, è il servizio di hosting, che in questo caso viene offerto da Flask, un microframework dedito allo sviluppo di applicazioni web con l'ausilio del linguaggio di programmazione Python. Flask risulta molto favorevole per la configurazione di un server web e per lo sviluppo di template per pagine web, i quali vengono utilizzati per poter usufruire dei dati, del codice in back-end, e metterli a disposizione sulla pagina web. Si chiarisce che i suddetti template, che permettono la creazione di pagine dinamiche, vengono elaborati dal motore di template Jinja2, il quale prevede una specifica sintassi.

Nell'applicazione viene definito un modulo **gui**, che predispone l'istanza Flask *app*, la quale verrà sfruttata per accedere alla pagina web creata, però prima di attivare a tutti gli effetti il server web, si dovranno predisporre le variabili da fornire al template e viene fatto nel modo seguente:

```
1 from flask import Flask, render_template
2 import json
3
4 app = Flask(__name__)
5
6 @app.route('/')
7 def index():
8     target = app.config.get('TARGET')
9     data = app.config.get('DATA')
10    network_routers = app.config.get('NETWORK_ROUTERS_JSON')
```

¹Per la spiegazione dell'applicazione consultare la fonte

```

11
12     return render_template('index.html', target=target, data=data
13     , network_routers=network_routers)
14
15 if __name__ == '__main__':
16     app.run(host='0.0.0.0', port=5000, debug=True)

```

Listing 5.1: Modulo gui

Nel codice, le variabili sono **target**, **data** e **network_routers** e verranno passate come parametri della funzione *render_template*, che si occuperà di costruire la pagina web utilizzando il template **index.html**, con l'ausilio appunto delle variabili dette sopra. Allo scopo di indicare con maggiore dettaglio il funzionamento di Jinja2, si riporta il codice presente nel file html:

```

1 let router = "{{ target|safe }}";
2 let networkData = {{data|safe}};
3 let networkRouters = {{ network_routers|safe }};

```

Listing 5.2: Sintassi Jinja2 per parametri

Da come si può notare, il placeholder `{{ }}`, proprio della sintassi Jinja2, verrà sostituito con la rispettiva variabile indicata, in precedenza passata come parametro. Per evitare di compromettere il risultato html, tramite caratteri eventualmente presenti nei parametri, si effettua l'operazione di escape usando il costrutto `|safe`. Prima di passare oltre questo argomento, si richiama il codice del file main, dedicato a preparare i parametri presenti nelle chiamate del metodo *get*.

```

1 network_routers_json = {router_id: json.loads(router.toJSON())
2   for router_id, router in network_routers.items()}
3
4 data = network_topology.toJSON()
5
6 app.config['NETWORK_ROUTERS_JSON'] = network_routers_json
7 app.config['DATA'] = data
8 app.config['TARGET'] = input_node
9
10 flask_thread = Thread(target=run_flask, daemon=True)
11 flask_thread.start()

```

Listing 5.3: Gestione processo Flask in main.py

Come osservabile da questo blocco di codice, per poter passare dei parametri all'istanza Flask, si dovranno creare degli elementi, del tipo chiave valore, da inserire in *config*. Si spiega brevemente lo scopo di ogni parametro presente:

- **Network_Routers_Json:** struttura dati in cui sono presenti le istanze Node, in formato JSON, di tutti i nodi della topologia di rete.
- **Data:** contiene l'istanza di tipo Network, in formato JSON, che contiene tutte le informazioni sulla topologia di rete

- **Target:** contiene il router id del nodo scelto (nodo target)

Per poter gestire le richieste, effettuate tramite richieste HTTP al localhost, viene utilizzato un thread, che permette una gestione parallela e leggera del nostro web server. Il codice, che dovrà essere eseguito dal thread, viene indicato dal parametro **target** e solamente quando il thread terminerà la propria esecuzione, allora anche il processo principale potrà fare altrettanto, ciò è reso possibile dal parametro *daemon* settato a True.

5.2 Vis.js

Spendiamo qualche parola per descrivere la libreria vis.js. La libreria vis.js è una libreria open-source scritta con il linguaggio javascript, che permette la visualizzazione di dati dinamici, i quali possono essere di varia natura come: grafi, reti, timeline ecc. Per gli scopi dell'applicazione, si farà uso della componente dedicata alla rappresentazione di reti, denominata *Network*, tramite la quale potremo definire nodi e archi, che rispettivamente rappresenteranno i nodi e link della topologia di rete. Questi elementi verranno inseriti nelle due strutture dati **nodes** e **edges**, in quanto dati necessari da passare, come parametri, al costruttore *Network*. Questi passaggi, spiegati fino ad ora, si possono ritrovare nel codice presente nel template index, di seguito, quindi, si riportano degli spezzoni di codice interessati:

```
1 //parametro options
2 let options = {
3     nodes: {
4         shape: 'image',
5         size: 40,
6         font: { size: 21 }
7     },
8     edges: {
9         smooth: false,
10        font: { size: 17, align: 'top' }
11    },
12    layout: {
13        randomSeed: 2
14    },
15    interaction: {
16        hover: true,
17        zoomView: false
18    },
19    physics: {
20        enabled: false
21    }
22 };
23
24 //esempio inserimento di un nodo
25 nodes.push({
26     id: switchId,
27     label: "SW" + swCounter,
28     image: switchImage
29 });
```

```
30
31 //esempio inserimento di un arco
32 edges.push({
33     from: switchId,
34     to: endpoints[i],
35     label: link.id + " / " + find_subnet_length(link.prefix),
36     title: link.id,
37     arrows: 'none',
38     color: { color: areaColor }
39 });
40
41 //costruttore vis.Network
42 let container = document.getElementById('network');
43 let data = { nodes: nodes, edges: edges };
44 let network = new vis.Network(container, data, options);
```

Listing 5.4: Codice javascript Vis.js

5.3 Interfaccia interattiva

In aggiunta a tutto ciò che è stato detto, il template definisce anche degli eventHandler di tipo click per alcuni elementi della pagina web: router, link e area id. Ogni handler fornisce in output i dati raccolti sull'elemento cliccato, in particolare:

- se l'elemento è un router, verranno stampate le seguenti informazioni: hostname, interfacce IPv6, vicini OSPFv3 e route table IPv6 (esempio di output in figura 5.3).
- se l'utente clicca su un link, in output si otterrà: prefisso, lunghezza del prefisso, tipo link, metrica, endpoint ed eventualmente DR e BDR, questo vale solo per link transitNetwork (esempio in figura 5.3).
- se viene cliccato l'area id invece verranno mostrate le rotte inter-area e le rotte esterne che vengono pubblicizzate nell'area in questione (esempio in figura 5.3).

Per concludere l'insieme delle informazioni offerte, l'interfaccia grafica mostra anche le rotte esterne pubblicizzate all'interno dell'AS, si riporta un esempio di come viene indicata in figura 5.3

5.4 differenze per reti IPv6

Rispetto al template usato per OSPFv2, quello utilizzato nell'applicazione mostra delle differenze, che verranno dette successivamente, anche con lo scopo di evidenziare le differenze di dati presentati con il cambio versione del protocollo. La prima differenza consiste nelle informazioni sui vicini: dato che il comando utilizzato per ottenere le informazioni dai vicini OSPFv3 non fornisce alcun indirizzo IPv6, è stato modificato l'output del handler click effettuato su un router, eliminando la parte dell'indirizzo. Un'ulteriore differenza è riscontrata nella procedura per ottenere la lunghezza della subnet, nella quale viene effettuato unicamente lo split per ottenere la lunghezza del prefisso considerato.

Hostname: C

Router ID: 10.0.0.3

Interfaces:

- **Ethernet1**
IP: fe80::a8c1:abff:fea8:f85b **Prefix length:** 64
IP: fd00:11::2 **Prefix length:** 64
Status: connected **Line Protocol:** up
- **Ethernet2**
IP: fe80::a8c1:abff:feb1:4841 **Prefix length:** 64
IP: fd00:22::2 **Prefix length:** 64
Status: connected **Line Protocol:** up
- **Ethernet3**
IP: fe80::a8c1:abff:fe70:5f9f **Prefix length:** 64
IP: fd00:c::1 **Prefix length:** 64
Status: connected **Line Protocol:** up
- **Management0**
IP: fe80::7441:d9ff:febf:4a7a **Prefix length:** 64
IP: 3fff:172:20:20::4 **Prefix length:** 64
Status: connected **Line Protocol:** up

Neighbors:

- **Router ID:** 10.0.0.1 **Interface:** Ethernet1 **Adjacency State:** full
- **Router ID:** 10.0.0.2 **Interface:** Ethernet2 **Adjacency State:** full

Figura 5.1: Esempio di output click nodo

Dettagli Link

IP: [fd00:a::]

Prefix length: 64

Tipo: stubNetwork

Metric: 10

Endpoints: 10.0.0.1

Figura 5.2: Esempio di output click nodo

Dettagli Link**IP:** [fd00:11::]**Prefix length:** 64**Tipo:** transitNetwork**Metric:** 10**Designated Router:** 10.0.0.3**Backup Designated Router:** 10.0.0.1**Endpoints:** 10.0.0.3, 10.0.0.1

Figura 5.3: Esempio di informazioni transit link

Rotte Inter-Area OSPF:

- Dest: fd00:e::/64 via 10.0.0.1,10.0.0.2

Percorsi verso ASBR:

- ASBR: 10.0.0.4 via 10.0.0.1 (metrica: 10)
- ASBR: 10.0.0.4 via 10.0.0.2 (metrica: 10)

Figura 5.4: Esempio di output click nodo

Rotte Esterne:

- Dest: ::/0 via 10.0.0.1 (metrica: 1, tipo: type2)

Figura 5.5: Esempio di output click nodo

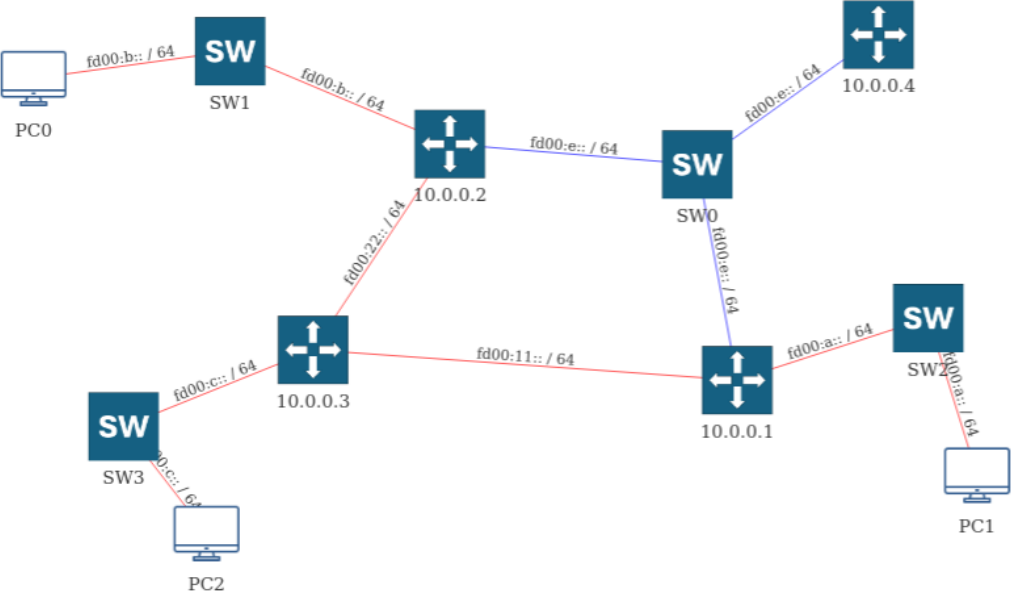


Figura 5.6: Grafo costruito nel primo caso di test

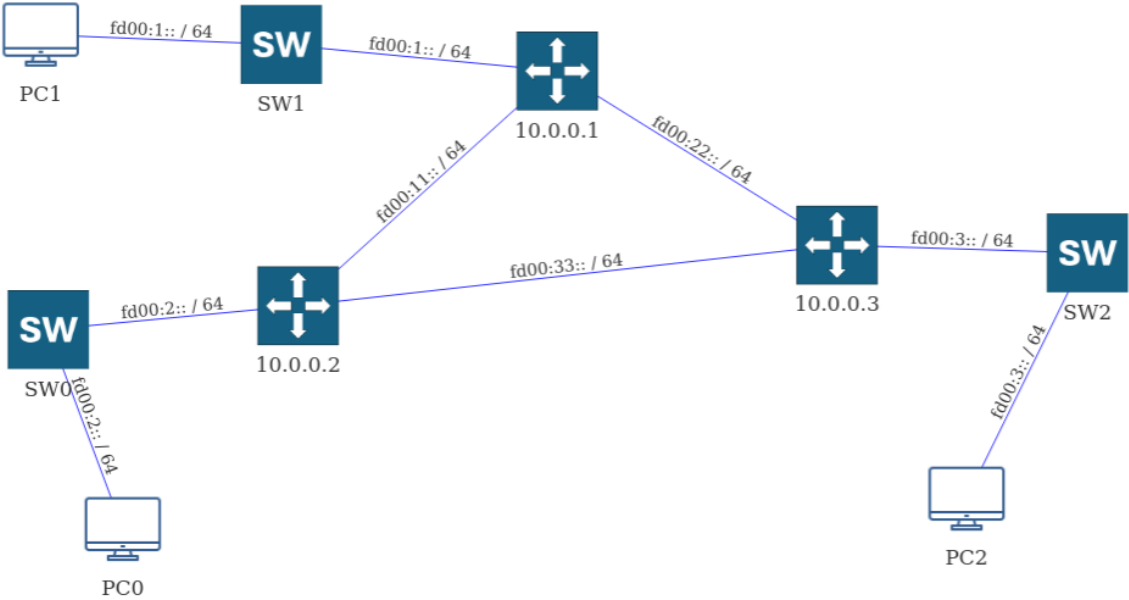


Figura 5.7: Grafo costruito nel secondo caso di test

Capitolo 6

Utilizzo dell'applicazione

Conclusa la trattazione sullo sviluppo dell'applicazione, si procede ad illustrare le indicazioni per utilizzare l'applicazione e introdurre i casi di test. Fase preliminare, a quella che si ritroverà successivamente, è quella di aver predisposto tutti gli strumenti e aver compreso i concetti teorici, in maniera tale che si possa avere piena conoscenza di ciò che l'applicazione fa e le informazioni rappresentate. In aggiunta verranno dettagliati maggiormente i comandi offerti all'utente dal terminale dell'applicazione, in modo tale da fornire direttive il più complete possibili sull'utilizzo dell'applicazione.

6.1 Script bash

Per supportare l'avvio e la configurazione dell'applicazione, sono presenti due script bash, tramite i quali sarà possibile avviare l'applicazione e configurare dei bridge sulla macchina host. Partiamo dal primo di questi: il primo script ha il compito di avviare l'applicazione python, in un ambiente virtuale (venv), utilizzando l'indirizzo IPv4 (interfaccia di loopback) passato come parametro, si riporta di seguito il suddetto:

```
1 ip=$1
2
3 source venv/bin/activate
4
5 python3 src/main.py "$ip"
```

Listing 6.1: Script per avviare l'applicazione

Prima di passare al secondo script, si deve spiegare innanzitutto il motivo della sua implementazione ed è il seguente: emulare gli switch attraverso i nodi Arista comporta un costo computazionale non trascurabile, il quale rischia di non fare terminare con successo il deploy della topologia. Lo scopo del secondo script è, quindi, quello di creare dei bridge nell'host e attivarli, per poterli poi utilizzare nella topologia¹

```
1 #!/bin/bash
2
3 switch_list=$1
```

¹Si ricorda il type bridge introdotto nel paragrafo [3.2](#)

```
4
5 for elemento in $switch_list; do
6     #creo il bridge
7     ip link add $elemento type bridge
8     #lo attivo
9     ip link set $elemento up
10 done
```

Listing 6.2: Script di attivazione bridge

6.2 Immagini dei container

Nell'applicazione verrà adottata un'immagine **Arista** per i router, in particolare è stata scelta l'immagine **ceos:4.34.0F**, la quale è ottenibile dal sito ufficiale di Arista e permette di offrire le funzionalità al livello di rete richieste. Invece per gli endpoint viene utilizzata un'immagine personalizzata, che dispone i componenti necessari per operare con l'applicazione, ed è realizzata tramite un **Dockerfile** illustrato di seguito:

```
1 # Parto da un'immagine python che contiene
2 # librerie installate anche per l'uso di firefox
3 FROM python:3.12.3-slim
4 # faccio update e installo tutte le dipendenze
5 # necessarie per poter utilizzare firefox
6 RUN apt-get update && apt-get install -y firefox-esr curl wget
   gnupg && rm -rf /var/lib/apt/lists/*
7 # Installo pyeapi e Flask
8 RUN pip install --no-cache-dir flask pyeapi
9
10 # elementi per configurazione della scheda di rete
11 RUN apt-get update && apt-get install -y iproute2 iputils-ping
   net-tools && rm -rf /var/lib/apt/lists/*
12
13
14 # Espone la porta per Flask per poter accedere
15 # al template
16 EXPOSE 5000
```

Listing 6.3: Dockerfile immagine personalizzata

6.3 Terminale dell'applicazione

Una volta che i preparativi saranno ultimati e aver effettuato il deploy della topologia, sarà possibile avviare l'applicazione tramite lo script **ospf_mgmt.sh**, scegliendo un router tra quelli presenti nella topologia. L'output mostrato inizialmente, prevede la stampa delle informazioni proprie del nodo target, come mostrato in seguito: Terminata la stampa delle suddette

```

root@H1:/AutoNetMaster/AutoNetmaster# ./ospf_mgmt.sh 10.0.0.1
**** OSPFv3 Management APP ****

Hostname: A
Router ID: 10.0.0.1
Interfaces:
  ID: Ethernet1,
    address:fe80::a8c1:abff:fe06:695f/64, type:link local
    address:fd00:e::1/64, type:config
    Interface Status: connected, Line Protocol Status: up
  ID: Ethernet2,
    address:fe80::a8c1:abff:fea4:8e0f/64, type:link local
    address:fd00:11::1/64, type:config
    Interface Status: connected, Line Protocol Status: up
  ID: Ethernet3,
    address:fe80::a8c1:abff:fe0c:7d49/64, type:link local
    address:fd00:a::1/64, type:config
    Interface Status: connected, Line Protocol Status: up
  ID: Management0,
    address:fe80::ec4e:36ff:fe39:e141/64, type:link local
    address:3fff:172:20:20::3/64, type:config
    Interface Status: connected, Line Protocol Status: up
Neighbors:
  Interface: Ethernet1, Router ID: 10.0.0.4, Adj-State: full, DR: 10.0.0.4, BDR: 10.0.0.2
  Interface: Ethernet1, Router ID: 10.0.0.2, Adj-State: full, DR: 10.0.0.4, BDR: 10.0.0.2
  Interface: Ethernet2, Router ID: 10.0.0.3, Adj-State: full, DR: 10.0.0.3, BDR: 10.0.0.1
Route Table:
  Destination: 3fff:172:20:20::/64
    Via: Directly connected, Interface: Management0
    Protocol: connected
  Destination: fd00:a::/64
    Via: Directly connected, Interface: Ethernet3
    Protocol: connected
  Destination: fd00:b::/64
    Via: fe80::a8c1:abff:fea8:f85b, Interface: Ethernet2
    Protocol: ospf
  Destination: fd00:c::/64
    Via: fe80::a8c1:abff:fea8:f85b, Interface: Ethernet2
    Protocol: ospf

```

Figura 6.1: Apertura applicazione

informazioni, il programma attenderà un input dall'utente, dove quest'ultimo potrà inviare uno dei seguenti comandi:

- **id [ospf_id]:** Comando che realizza la stampa a video delle informazioni di un router della rete, scelto dall'utente specificando il relativo router id dell'istanza OSPFv3. Il comando mostra un messaggio di errore nel caso in cui non venga trovato il router in questione.
- **nodes:** Comando che mostra le informazioni raccolte su tutti i router appartenenti alla topologia.
- **topology:** Effettua la stampa delle informazioni ottenute durante la ricostruzione della topologia di rete, evidenziando il contenuto di ogni area, specificando tutti i link e le rotte inter-area, passando poi alle rotte verso ASBR e le rotte esterne.
- **display:** Attiva il thread dedito alla gestione del server web Flask, il quale, nel momento in cui l'utente effettuerà la richiesta HTTP, preparerà il template.
- **help:** permette di mostrare a video una panoramica dei comandi disponibili.
- **exit:** termina l'applicazione.

Si vuole sottolineare un punto importante: dato che i container disposti non supportano un server grafico, per poter essere il più leggeri possibile, sarà necessario usare il browser dell'host per poter accedere all'interfaccia grafica web based, ciò invece non è necessario nel caso in cui l'applicativo venga eseguito su un normale host oppure in un container che fornisce i supporti grafici necessari.

Capitolo 7

Conclusioni

7.1 Considerazioni sul lavoro svolto

ContainerLab si è rivelato essere un software estremamente comodo per emulare le topologie di rete, ciò reso possibile anche grazie ai container, tramite i quali ho avuto la possibilità di creare nodi personalizzati e effettuare altri tipi di configurazioni, come le porte, file di configurazioni ecc. Inoltre, la scelta di immagini Arista si è rivelata vantaggiosa, in quanto ha fornito una CLI simile all'ambiente Cisco IOS (già noto), e ha permesso grande interattività con il programma. Quest'ultimo, scritto in Python, ha utilizzato il modulo pyeapi, con il quale si è riusciti ad effettuare connessioni con i nodi e inviarvi comandi, ricevendo response in formato JSON, che ha facilitato il recupero dei dati necessari. Nonostante ci sia stato il bisogno di adeguare l'applicazione persistente, per poter operare con il protocollo OSPFv3, si sono riscontrate svariate similitudini, ampiamente sfruttate per creare questa nuova versione. In aggiunta, Flask è stato alla base dell'interfaccia grafica web-based, realizzata con un template in grado di ricevere parametri dal programma principale. La scelta dell'approccio via web si è rivelato vantaggioso, in quanto ha sfruttato delle componenti offerte da javascript e Vis.js, libreria semplice ed efficace, e non ha richiesto particolari modifiche per essere adeguata a OSPFv3.

7.2 Possibilità future

Questa applicazione ha degli sbocchi futuri, che ne permettono ulteriore completezza. Uno tra questi potrebbe essere ipotizzare la presenza di NSSA (Not-So-Stubby-Area), che nella applicazione si è supposto non essere presenti per semplicità; un'altro aspetto potrebbe invece essere la gestione di più istanze OSPFv3, che potrebbe generare le diverse topologie presenti nella rete. Inoltre per questa applicazione è stato adottato il protocollo OSPFv3, però potrebbe essere riadattata per poter gestire RIP per IPv6 oppure altri protocolli di rete che richiedono gestione da parte degli sviluppatori di rete.

Bibliografia

- [1] Arista Networks. Documentazione ufficiale arista. <https://www.arista.com/en/firmware/eos>.
- [2] Arista Networks. pyeapi documentation. <https://pyeapi.readthedocs.io/en/latest/>.
- [3] Containerlab Project. Containerlab - flexible networking lab deployment. <https://containerlab.dev>.
- [4] Silvia Hagen. *IPv6 Essentials*. O'Reilly Media, 2nd edition, 2014.
- [5] Pallets Projects. Flask documentation. <https://flask.palletsprojects.com>.
- [6] vis.js Contributors. vis.js network documentation. <https://visjs.github.io/vis-network>.